
Data Structures

BS (CS) _Fall_2025

LAB 01 Tasks



Lab Overview

This lab focuses on understanding and implementing function templates in C++ along with unit testing using Google Tests. Students will start with a brief introduction of templates and progress to implementing functions using templates and validating them using Google Tests.

Task 0: Template Function Usage (Warm-up)

Difficulty: Beginner

Objective: Get familiar with calling template functions

Given Code:

Students are provided with the following template functions:

```
template<typename T>
T add(T a, T b) {
    return a + b;
}

template<typename T>
T subtract(T a, T b) {
    return a - b;
}

template<typename T>
T multiply(T a, T b) {
    return a * b;
}

template<typename T>
T divide(T a, T b) {
    return a / b;
}
```

Student Task:

Write a `main()` function that:

1. Tests each function with at least 3 different data types (int, float, double)
2. Demonstrates both explicit template instantiation (`add<int>(5, 3)`) and implicit instantiation (`add(5, 3)`)
3. Includes proper error handling for division by zero
4. Prints results in a formatted manner

Task 1:

Convert given fixed-type functions into function templates, test them, and where needed, add special handling for specific datatypes.

Add Two Integers:

Function:

```
int addInt(int a, int b) {  
    return a + b;  
}
```

Convert this into a generic template function `add<T>` so it works for `int`, `float`, and `double`.

Test Cases:

- Add integers: `add(3, 7)` 10.
- Add doubles: `add(2.5, 3.1)` 5.6.
- Add floats: `add(1.1f, 2.2f)` 3.3f.
- Add strings (concatenation): `add("Hello", " World")` → "Hello World"

Swap Two Characters

```
void swapChar(char &a, char &b) {  
    char temp = a;  
    a = b;  
    b = temp;  
}
```

Make this a template `swap<T>`, so it works for `int`, `float`, `char`, etc.

Test Cases:

- Swap two integers: `a=5, b=10`, after swap `a=10, b=5`.
- Swap two chars: `a='x', b='y'`, after swap `a='y', b='x'`.
- Swap two doubles: `a=1.1, b=2.2`, after swap `a=2.2, b=1.1`.

Division of Integers

```
int divideInt(int a, int b) {  
    if (b == 0) return -1;  
    return a / b;  
}
```

Convert into a template `divide<T>`

- For floating point, perform normal division.
- For integer, check denominator = 0, return -1.

Test Cases:

- Divide integers: `divide(10, 2) → 5`.
- Integer divide by zero: `divide(10, 0) → -1`.
- Divide doubles: `divide(10.0, 2.0) → 5.0`.
- Floating divide by zero: `divide(10.0, 0.0) → inf` or handled gracefully.

Task 2: Basic Template Function Implementation

Difficulty: Easy

Objective: Write simple template functions

Requirements:

Implement the following template functions:

1. `findMax(T a, T b)` - Returns the maximum of two values
2. `findMax(T1 a, T2 b)` - Returns the maximum of two values
3. `findMax(float a, double b)` - Returns the maximum of two values
4. `swap(T& a, T& b)` - Swaps two values (by reference)
5. `absolute(T value)` - Returns absolute value
6. `absolute(double value)` - Returns absolute value

Task 3: Template Functions with Strings

Difficulty: Easy

Objective: Create template functions that work with different string types

Requirements:

Implement the following template functions that work with string types:

1. `stringLength(const T& str)` - Returns the length of any string-like object
2. `concatenateStrings(const T& str1, const T& str2)` - Concatenates two strings of the same type
3. `reverseString(T str)` - Reverses any string-like object
4. `reverseString(char* str)` - Reverses the string
5. `convertCase(T str, bool toUpper)` - Converts string to uppercase or lowercase based on boolean flag
6. `findsubstring(T str, string substr)` - Check if the substring exists, if yes then return the starting index of the substring
7. `findsubstring(T str, char* substr)` - Check if the substring exists, if yes then return the starting index of the substring

Task 4: Template Functions with Arrays

Difficulty: Medium

Objective: Create template functions that work with arrays and perform various operations

Requirements:

Implement the following template functions that work with arrays:

1. `findMinInArray(T arr[], size_t size)` - Find the minimum element in an array
2. `searchArray(T arr[], size_t size, const T& target)` - Linear search for an element, return index or -1
3. `reverseArray(T arr[], size_t size)` - Reverse the elements of an array in-place
4. `arraySum(T arr[], size_t size)` - Calculate sum of array elements (for numeric types)

Hint:

Size of an array can be found using: `size_t size = sizeof(arr) / sizeof(arr[0]);`

Task 5: Unit Testing with Google Test

Difficulty: Medium

Objective: Write comprehensive unit tests for all previous tasks

Requirements:

Create a comprehensive test suite using Google Test framework that covers:

Required Test Cases for Task 2:

1. **findMax(), swap(), absolute() Tests:**
 - Test with integers, floats, doubles
 - Test with positive and negative numbers
 - Test with zero
 - Test with equal values
 - Test with characters
 - Also handle edge cases

Required Test Cases for Task 3:

1. **stringLength() Tests:**
 - Test with normal strings
 - Test with empty strings
2. **concatenateStrings() Tests:**
 - Test string concatenation

- Test with empty strings
- Test with special characters
- 3. **reverseString() Tests:**
 - Test normal strings
 - Test single character strings
 - Test empty strings
- 4. **convertCase() Tests:**
 - Test uppercase conversion
 - Test lowercase conversion
 - Test mixed case strings
 - Test strings with special chars
- 5. **findsubstring() Tests:**
 - Test normal strings
 - Test single character strings
 - Test empty strings
 - Test strings with special chars

Required Test Cases for Task 4:

Required Test Cases:

1. **findMinInArray() Tests:**
 - Test with different array sizes
 - Test with sorted/unsorted arrays
 - Test with duplicate minimum values
 - Test with different data types
2. **searchArray() Tests:**
 - Test successful searches
 - Test unsuccessful searches
 - Test with empty arrays
 - Test with duplicate elements
3. **reverseArray() Tests:**
 - Test with odd/even sized arrays
 - Test with single element arrays
 - Test with different data types
4. **arraySum() Tests:**
 - Test with positive numbers
 - Test with negative numbers
 - Test with mixed positive/negative
 - Test with floating-point arrays